

# COL851 A1

Sarthak Panda (2022CS51217)

## System Hardware Configuration

Utilized `lscpu` and `free -h` to get the Hardware Details [Note: Inferences were run on WSL2]

| Field               | Value                                                                                                              |
|---------------------|--------------------------------------------------------------------------------------------------------------------|
| Architecture        | x86_64                                                                                                             |
| CPU op-mode(s)      | 32-bit, 64-bit                                                                                                     |
| Address sizes       | 46 bits physical, 48 bits virtual                                                                                  |
| Byte Order          | Little Endian                                                                                                      |
| CPU(s)              | 22                                                                                                                 |
| On-line CPU(s) list | 0-21                                                                                                               |
| Vendor ID           | GenuineIntel                                                                                                       |
| Model name          | Intel(R) Core(TM) Ultra 9 185H                                                                                     |
| CPU family          | 6                                                                                                                  |
| Model               | 170                                                                                                                |
| Thread(s) per core  | 2                                                                                                                  |
| Core(s) per socket  | 11                                                                                                                 |
| Socket(s)           | 1                                                                                                                  |
| Stepping            | 4                                                                                                                  |
| BogoMIPS            | 6143.99                                                                                                            |
| Virtualization      | VT-x                                                                                                               |
| Hypervisor vendor   | Microsoft                                                                                                          |
| Virtualization type | full                                                                                                               |
| Caches (sum of all) | L1d: 528 KiB (11 instances)<br>L1i: 704 KiB (11 instances)<br>L2: 22 MiB (11 instances)<br>L3: 24 MiB (1 instance) |
| NUMA node(s)        | 1                                                                                                                  |
| NUMA node0 CPU(s)   | 0-21                                                                                                               |
| Total installed RAM | 15 GiB                                                                                                             |
| Swap total          | 4.0 GiB                                                                                                            |

## LLM models used for running inference with `llama.cpp`

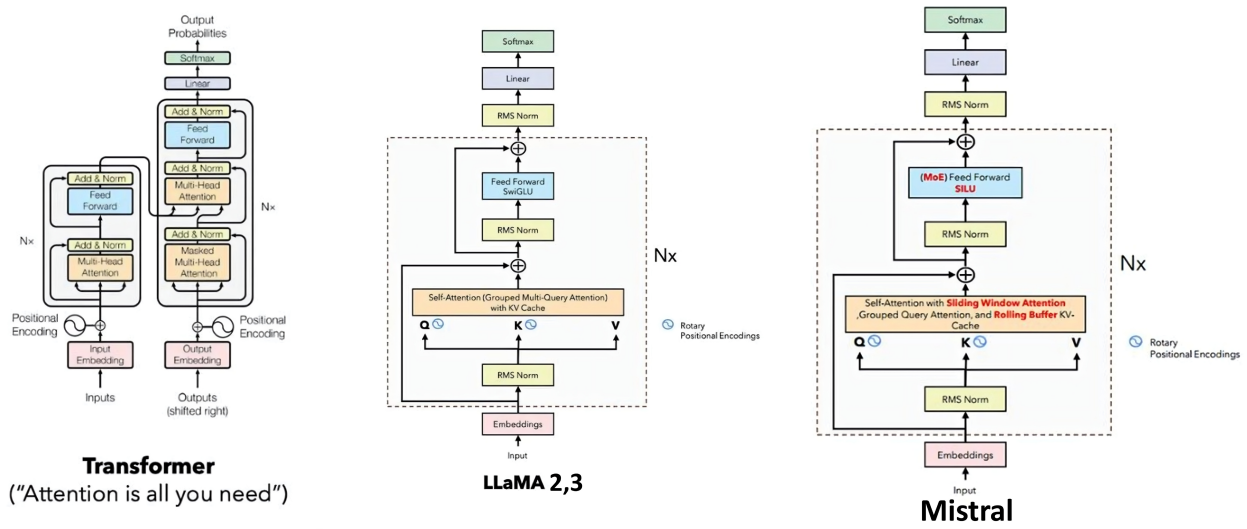
Note: All models are in GGUF format

| Model                  | Link                      | Num<br>Params. | Num<br>MAC ops. | Memory<br>footprint | Avg. ( $\mu$ ) Inference<br>Latency (100 runs) | Std. Dev.<br>( $\sigma$ ) |
|------------------------|---------------------------|----------------|-----------------|---------------------|------------------------------------------------|---------------------------|
| 7b-llama2-q4_0         | <a href="#">Link Here</a> | ~6.74B         | ~6.73B/token    | ~3.56GB             | 23.95s                                         | 7.66s                     |
| mistral:7b-text-q5_K_M | <a href="#">Link Here</a> | ~7.24B         | ~7.11B/token    | ~4.78GB             | 28.10s                                         | 2.10s                     |
| llama2:13b-text-q4_K_M | <a href="#">Link Here</a> | ~13.02B        | ~14.53B/token   | ~7.33GB             | 42.24s                                         | 14.20s                    |

[Link to python script is Here.](#)

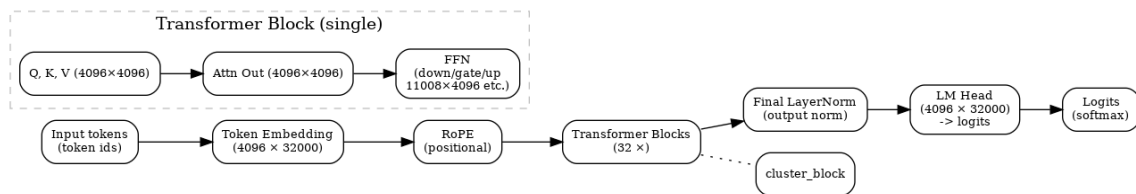
# Examined LLM Model's Architecture

As mentioned in above Table we have used the models like "llama2", "mistral". All of them have same general architecture of llama i.e. they all of them are some sort of modification on base llama model. The following image from the Medium article helps to identify the key differences among these Architectures.

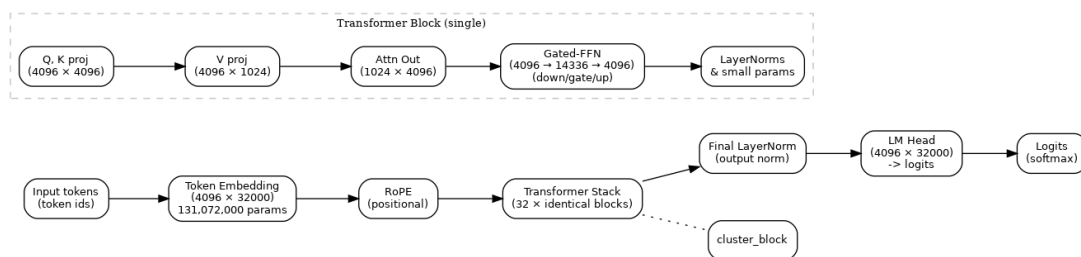


| Feature                   | Plain Transformers | LLaMA 3 and 2                 | Mistral                         |
|---------------------------|--------------------|-------------------------------|---------------------------------|
| Architecture              | Encoder-Decoder    | Single Layer                  | Single Layer + Enhancements     |
| Positional Encoding       | Standard           | Rotary                        | Rotary + Additional Innovations |
| Normalization             | LayerNorm          | RMS Norm                      | RMS Norm + Enhanced Mechanisms  |
| Attention Mechanism       | Self-Attention     | Grouped Multi-Query Attention | Sliding Window Attention        |
| Memory Efficiency         | N/A                | KV Cache                      | Rolling Buffer KV-Cache         |
| Activation in Feedforward | RELU               | SwiGLU                        | SwiGLU + Mixture of Experts     |

A more detailed layer wise architecture of these model is available on the above link in the Table. For llama-2 7b a simple block diagram of architecture with few more additional details is provided below.



A similar block diagram can be made for the case of Mistral 7b model.



# LLaMA.cpp Installation and Environment Setup

The following commands will install LLaMA.cpp and set up the required Python environment on Ubuntu/Debian systems:

```
# Step 1: Update system packages and install build dependencies
sudo apt update && sudo apt install -y build-essential git cmake python3 python3-venv
python3-pip curl

# Step 2: Create project directory and navigate to it
mkdir -p ~/llm-local && cd ~/llm-local

# Step 3: Clone the LLaMA.cpp repository and enter the directory
git clone https://github.com/ggerganov/llama.cpp.git && cd llama.cpp

# Step 4: Create build directory and configure the project
mkdir -p build && cd build

# Step 5: Configure CMake with Release build type (CURL disabled for cleaner build)
cmake .. -DCMAKE_BUILD_TYPE=Release -DLLAMA_CURL=OFF

# Step 6: Build the project using all available CPU cores
cmake --build . -j$(nproc)

# Step 7: Return to project root and create Python virtual environment
cd ../.. && python3 -m venv venv && source venv/bin/activate

# Step 8: Upgrade pip and install Python dependencies
python -m pip install --upgrade pip && pip install llama-cpp-python tqdm datasets
```

To activate the Python environment in future sessions, navigate to the project directory and run:

```
source venv/bin/activate
```

We can manually download the model from the above links provided and place them in the working directory or use curl command to get this done. In any case make sure that the final working directory looks as shown below:

```
llm-local
├> llama.cpp
├> venv
├> batch_infer.py
├> prompts.txt
├> model1.gguf
└> (other models...)
```

The following is a minimal python script for batch\_infer.py that demonstrates how to use the llama-cpp-python library to process multiple prompts and save results. Finally, to run inferences execute `python3 ~/llm-local/batch_infer.py`.

```
from llama_cpp import Llama
import time, json, os
model_path = os.path.expanduser('~/.llm-local/model1.gguf')
llm = Llama(model_path=model_path)
with open(os.path.expanduser('~/.llm-local/prompts.txt')) as f, \
    open(os.path.expanduser('~/.llm-local/outputs.jsonl'), 'w') as out:
    for i, p in enumerate(f):
        p = p.strip()
        if not p: continue
        t0 = time.time()
        res = llm.create_completion(prompt=p, max_tokens=128)
        t1 = time.time()
        output_data = {
            'id': i,
            'latency': t1 - t0,
            'output': res['choices'][0]['text']
        }
        out.write(json.dumps(output_data) + '\n')
```